

Logic and Proof

Algorithms How-To + One-Page Rule Reference

Part IB — Cambridge CST — Paper 6

What the exam actually asks

Logic & Proof is an **algorithm-running** course: questions hand you a formula or clause set and say “apply DPLL / resolution / build the BDD.” Across **2018–2025 (16 questions)**:

Topic	Appearances	Share
Resolution theorem proving	7	44%
Binary Decision Diagrams	6	38%
DPLL method	5	31%
First-order logic / Skolemisation & unification	3	19%
Sequent calculus	3	19%
CNF conversion; Fourier–Motzkin; model checking; S4/S5 modal	2 each	12%

80/20 verdict. Resolution, BDDs and DPLL are the spine (~60% of marks) — you must be able to *run each by hand, showing every step*. They all start from **clause/CNF form**, so master that conversion first. Then sequent calculus + quantifier rules, unification, Fourier–Motzkin, and modal S4. This guide gives the **recipe + a worked example** for each algorithm, then a **one-page rule reference** (last page) to memorise.

1 Clause / CNF conversion (everything starts here)

Algorithm

[title=Propositional CNF (4 steps)]

1. **Eliminate** $\leftrightarrow, \rightarrow$: $A \leftrightarrow B \simeq (A \rightarrow B) \wedge (B \rightarrow A)$, $A \rightarrow B \simeq \neg A \vee B$.
2. **Push** $\neg$ **inward** (to atoms) $\Rightarrow$ **NNF**: $\neg\neg A \simeq A$, de Morgan $\neg(A \wedge B) \simeq \neg A \vee \neg B$, etc.
3. **Push** $\vee$ **inward** (distribute over $\wedge$): $A \vee (B \wedge C) \simeq (A \vee B) \wedge (A \vee C)$.
4. **Simplify**: drop a disjunction with P and $\neg P$ ($\simeq \mathbf{t}$); $(P \vee A) \wedge (\neg P \vee A) \simeq A$; remove repeats.

A **clause** is a disjunction of literals, written as a set $\{\neg K_1, \dots, L_1, \dots\}$; the empty clause $\square = \{\}$ is **false** (contradiction).

Algorithm

[title=First-order clause form (for resolution)]

1. **Negate** the goal (resolution and DPLL are *refutation* methods — prove unsatisfiability).
2. Eliminate $\leftrightarrow, \rightarrow$; push $\neg$ in to get **NNF**.
3. **Skolemise**: for each $\exists y$ inside $\forall x_1 \dots \forall x_k$, replace y by $f(x_1, \dots, x_k)$ for a *new* f (a Skolem *constant* if no enclosing $\forall$). Push quantifiers *in* first (miniscope) $\Rightarrow$ fewer/smaller Skolem args $\Rightarrow$ shorter proofs.
4. **Drop the** $\forall$'s (clauses are implicitly universally quantified).
5. Distribute to **CNF**; write each conjunct as a clause (set of literals).

Skolemisation preserves *(un)satisfiability*, not validity — which is exactly what refutation needs.

2 DPLL (propositional SAT) — 31%

Algorithm

Refute a clause set (show unsatisfiable). Repeat:

1. **Delete tautologies** — any clause containing P and $\neg P$.
2. **Unit propagation** — for each unit clause $\{L\}$: delete every clause containing L , and delete $\neg L$ from all clauses.
3. **Pure-literal deletion** — if L occurs but $\neg L$ never does, delete all clauses containing L .
4. **Stop:** empty clause $\square$ generated $\Rightarrow$ **unsatisfiable** (refutation found). All clauses deleted $\Rightarrow$ **satisfiable**.
5. **Case split** on a literal L : recurse on the set $+\{L\}$ and on the set $+\{\neg L\}$. Satisfiable iff *either* branch is.

Worked: is $P \vee Q \rightarrow Q \vee R$ a theorem?

Negate + CNF gives clauses $\{P, Q\}, \{\neg Q\}, \{\neg R\}$.

$$\begin{array}{llll} \{P, Q\} & \{\neg Q\} & \{\neg R\} & \text{initial} \\ \{P\} & & \{\neg R\} & \text{unit } \neg Q \\ & & & \text{units } P, \neg R \text{ (also pure)} \Rightarrow \text{all deleted} \end{array}$$

All clauses deleted $\Rightarrow$ **satisfiable** ($P \mapsto 1, Q \mapsto 0, R \mapsto 0$), so the original formula is **not valid**. (If a case split is needed, show both $L/\neg L$ columns; reaching $\square$ in *every* branch = unsatisfiable = theorem.)

3 Resolution — 44% (the headline algorithm)

Algorithm

[title=Resolution refutation] To prove A : (1) negate A ; (2) convert $\neg A$ to clauses; (3) repeatedly apply the **resolution rule** to clauses with complementary literals; (4) derive $\square \Rightarrow \neg A$ unsatisfiable $\Rightarrow A$ is a theorem.

$$\frac{B \vee A \quad \neg B \vee C}{A \vee C} \quad \text{set form} \quad \frac{\{B, A_1, \dots, A_m\} \quad \{\neg B, C_1, \dots, C_n\}}{\{A_1, \dots, A_m, C_1, \dots, C_n\}}$$

Worked (propositional): prove $P \wedge Q \rightarrow Q \wedge P$

Negated clauses: $\{P\}, \{Q\}, \{\neg Q, \neg P\}$.

$$\frac{\{P\} \quad \{\neg P, \neg Q\}}{\{\neg Q\}} \quad \frac{\{Q\} \quad \{\neg Q\}}{\square}$$

$\square$ reached $\Rightarrow$ theorem. **Tip:** suppress repeated literals ($\{Q, Q\} = \{Q\}$); resolving a clause with its own tautology leads nowhere.

3.1 Unification (needed for first-order resolution)

Algorithm

[title=Unification algorithm (on term trees)] Find the **most general unifier** (MGU) θ with $t_1\theta = t_2\theta$. Cases:

- **variable x vs term t** : if $x = t$, return $[]$; **occurs check** — if x occurs in t , *fail*; else return $[t/x]$.
- **constant vs constant**: equal $\Rightarrow []$; differ $\Rightarrow$ fail.
- **pair (t_1, t_2) vs (u_1, u_2)** : unify t_1, u_1 as θ_1 ; unify $t_2\theta_1, u_2\theta_1$ as θ_2 ; return $\theta_1 \circ \theta_2$. Any failure $\Rightarrow$ not unifiable.

Worked unifications

$f(x, b)$ with $f(a, y)$: unify $x, a \Rightarrow [a/x]$; then $b, y \Rightarrow [b/y]$. **MGU** $[a/x, b/y]$.

$f(x, x)$ with $f(a, b)$: $[a/x]$ then a vs b **fail**.

$f(x, x)$ with $f(y, g(y))$: $[y/x]$ then y vs $g(y)$ — **occurs check fails**.

Exam trap

Skip the **occurs check** and you can “prove” non-theorems (it builds infinite terms / unsound proofs — this is why Prolog, which omits it, is not a sound theorem prover). **Rename variables apart** between the two clauses before resolving — but renaming is *not* part of unification itself.

3.2 First-order resolution & factoring

Algorithm

Binary resolution (with MGU σ of B and D):

$$\frac{\{B, A_1, \dots, A_m\} \quad \{\neg D, C_1, \dots, C_n\}}{\{A_1, \dots, A_m, C_1, \dots, C_n\}\sigma} \quad (B\sigma = D\sigma)$$

Factoring (needed for completeness — one resolution step removes only *one* complementary pair):

$$\frac{\{B_1, \dots, B_k, A_1, \dots, A_m\}}{\{B_1, A_1, \dots, A_m\}\sigma} \quad (B_1\sigma = \dots = B_k\sigma)$$

Worth knowing

Strategies: *set of support* — every step must involve (a descendant of) the negated conclusion. *Linear resolution* — each resolvent becomes a parent of the next step; this is the shape of **Prolog** execution (linear resolution on *Horn* clauses, left-to-right, depth-first). Delete *subsumed* and tautologous clauses to stop the prover flooding.

4 Binary Decision Diagrams — 38%

Algorithm

[title=ROBDD = canonical form] A BDD is an if-then-else DAG over the propositional variables. Write $x_P Y$ for “if P then X else Y ” (Shannon expansion). Three conditions make it canonical:

- **ordering** — variables tested in a fixed order; if P before Q then $P < Q$ (each var at most once per path);
- **uniqueness** — identical subgraphs stored once (hash by variable + child pointers);
- **irredundancy** — delete a node whose true- and false-children are identical.

Then: **equivalent** $\Leftrightarrow$ same BDD (compare pointers); **tautology** $\Leftrightarrow$ BDD = **1**; **unsatisfiable** $\Leftrightarrow$ BDD = **0**.

Algorithm

[title=Building the BDD of a connective (the “apply” recursion)] To combine $Z =_X P_Y$ and $Z' =_{X'} P'_Y$ under a connective:

1. recursively build BDDs for the sub-formulas, then combine top variables;
2. $P = P'$: new node on P with children $\text{apply}(X, X')$ and $\text{apply}(Y, Y')$;
3. $P < P'$: branch on P , pairing each child of P with the *whole* Z' (treat Z' as $_{Z'} P_{Z'}$); symmetric if $P > P'$;
4. **simplify** (uniqueness + irredundancy) at every step; if both children equal, return the child.

Use a **hash table** of (op, input BDDs) $\rightarrow$ result so each combination is computed once.

Exam trap

Variable ordering is everything: a good order gives a small BDD, a bad order can be exponentially larger. State the ordering you use and test variables in it consistently. Negation is cheap: copy the BDD and swap the **0/1** leaves.

5 Decision procedures: Fourier–Motzkin & SMT — 12%+

Algorithm

[title=Fourier–Motzkin variable elimination (linear arithmetic over $\mathbb{R}/\mathbb{Q}$)] Decide satisfiability of a conjunction of linear inequalities. To eliminate variable x :

1. rewrite each constraint mentioning x as a **lower bound** ($x \geq \ell$) or an **upper bound** ($x \leq u$);
2. add, for *every* (lower, upper) pair, the combined constraint $\ell \leq u$ (with x gone);
3. keep constraints not mentioning x ; **repeat** for the next variable.

End with a contradiction such as $0 \leq c$ with $c < 0 \Rightarrow$ **unsatisfiable**; otherwise satisfiable. (This is *quantifier elimination* for $\exists$.)

Worked: Fourier–Motzkin on a 4-constraint system

Constraints: $x \leq y$, $x \leq z$, $-x + y + 2z \leq 0$, $-z \leq -1$.

Third gives lower bound $x \geq y + 2z$; uppers are $x \leq y$, $x \leq z$. Pair them: $y + 2z \leq y \Rightarrow z \leq 0$;

$y + 2z \leq z \Rightarrow z \leq -y$. With $-z \leq -1$ (i.e. $z \geq 1$) we have $z \geq 1$ and $z \leq 0$: combine $\Rightarrow 0 \leq -1$.
Contradiction $\Rightarrow$ **unsatisfiable**.

Worth knowing

SMT = DPLL + theory solver. Treat each atomic arithmetic fact (e.g. $a < 0$) as a propositional letter; DPLL handles the Boolean structure, and a decision procedure checks whether the chosen literals are consistent in the theory. If not, it feeds back a *blocking clause* ($\neg(a < 0) \vee \neg(a > 0)$) and DPLL backtracks — **over-approximation + counterexample-driven refinement**. (Z3 is the canonical SMT solver.) *Note:* pure-literal deletion is unsafe in SMT — literals are connected through the theory.

6 Sequent calculus, modal logic & tableaux — 19%/12%

Algorithm

[title=How to do a sequent proof (work backwards)] Goal sequent $\Gamma \Rightarrow \Delta$ means “if all of Γ hold then some of Δ holds.” Start from the formula you want as $\Rightarrow A$ and apply rules **upwards**, breaking connectives down, until every leaf is a **basic sequent** $A, \Gamma \Rightarrow A, \Delta$ (same formula both sides). *Left* rules act on assumptions, *right* rules on goals (full set on the last page).

Exam trap

Apply the quantifier rules with a **proviso first**: $\forall r$ and $\exists l$ require “ x not free in the conclusion,” so use them *before* $\forall l/\exists r$ introduce terms that could capture x . $\forall l$ and $\exists r$ may be needed *twice* (keep a copy of the quantified formula). Rename to avoid clashes.

Worth knowing

Modal logic (Kripke). A frame is (W, R) : worlds + accessibility. $w \Vdash \Box A$ iff A holds in *all* v with $R(w, v)$; $w \Vdash \Diamond A$ iff in *some*. $\Diamond A \simeq \neg \Box \neg A$. Systems by axiom: **K** (distribution + necessitation), **T** $\Box A \rightarrow A$ (reflexive), **4** $\Box A \rightarrow \Box \Box A$ (transitive), **B** $A \rightarrow \Box \Diamond A$ (symmetric); **S4**=**T**+**4**, **S5**=**T**+**4**+**B** (equivalence relation). In **S4**, $\Box \Box A \simeq \Box A$ (operator strings collapse to length $\leq$ few). **S4 sequent rules** ($\Box r, \Diamond l$) *discard* all non- $\Box/\Diamond$ formulas — so apply them *first*, or you lose information and the proof fails.

Tableaux = a streamlined, refutation sequent calculus (only left rules; prove A by refuting $A \Rightarrow$). *Free-variable tableaux* add unification — but then you **must Skolemise** (mixing unification with un-Skolemised $\exists$ is unsound).

One-page rule reference

Propositional equivalences.

$\neg\neg A \simeq A$; $A \rightarrow B \simeq \neg A \vee B$; $A \leftrightarrow B \simeq (A \rightarrow B) \wedge (B \rightarrow A)$.

de Morgan: $\neg(A \wedge B) \simeq \neg A \vee \neg B$, $\neg(A \vee B) \simeq \neg A \wedge \neg B$.

distrib: $A \vee (B \wedge C) \simeq (A \vee B) \wedge (A \vee C)$ and dual.

simplify: $A \wedge \mathbf{t} \simeq A$, $A \vee \mathbf{f} \simeq A$, $A \wedge \neg A \simeq \mathbf{f}$, $A \vee \neg A \simeq \mathbf{t}$.

Clause form. negate $\rightarrow$ elim $\leftrightarrow$, $\rightarrow \rightarrow$ NNF $\rightarrow$ Skolemise ($\exists y$ under $\forall \bar{x} \mapsto f(\bar{x})$) $\rightarrow$ drop $\forall \rightarrow$ CNF $\rightarrow$ clauses. $\square = \{\} = \mathbf{f}$.

DPLL. del tautologies; unit-propagate $\{L\}$ (drop clauses with L , drop $\neg L$); del pure-literal clauses; $\square \Rightarrow$ unsat; empty set $\Rightarrow$ sat; else case-split on L .

Resolution. $\frac{\{B, \bar{A}\} \quad \{\neg D, \bar{C}\}}{\{\bar{A}, \bar{C}\}\sigma} (B\sigma = D\sigma)$.

Factoring $\frac{\{B_1, \dots, B_k, \bar{A}\}}{\{B_1, \bar{A}\}\sigma} (B_1\sigma = \dots = B_k\sigma)$.

Unification (MGU). var x vs t : occurs-check then $[t/x]$; const vs const: equal or fail; pair: unify firsts θ_1 , then $t_2\theta_1$ vs $u_2\theta_1$, compose.

BDD. $x_P Y \equiv$ if P then X else Y . Canonical when ordered + unique + irredundant. apply: $P = P'$ branch on P pairing children; $P < P'$ pair P 's children with all of Z' . Equiv $\Leftrightarrow$ same DAG; valid $\Leftrightarrow \mathbf{1}$; unsat $\Leftrightarrow \mathbf{0}$.

Fourier–Motzkin. per var: split into $x \geq \ell / x \leq u$; add $\ell \leq u$ for every pair; drop x ; repeat; $0 \leq c < 0 \Rightarrow$ unsat.

Sequent rules ($\Gamma \Rightarrow \Delta$; basic $A, \Gamma \Rightarrow A, \Delta$).

$$\begin{array}{c} \frac{\Gamma \Rightarrow \Delta, A}{\neg A, \Gamma \Rightarrow \Delta} (\neg l) \quad \frac{A, \Gamma \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, \neg A} (\neg r) \\[10pt] \frac{A, B, \Gamma \Rightarrow \Delta}{A \wedge B, \Gamma \Rightarrow \Delta} (\wedge l) \quad \frac{\Gamma \Rightarrow \Delta, A \quad \Gamma \Rightarrow \Delta, B}{\Gamma \Rightarrow \Delta, A \wedge B} (\wedge r) \\[10pt] \frac{A, \Gamma \Rightarrow \Delta \quad B, \Gamma \Rightarrow \Delta}{A \vee B, \Gamma \Rightarrow \Delta} (\vee l) \quad \frac{\Gamma \Rightarrow \Delta, A, B}{\Gamma \Rightarrow \Delta, A \vee B} (\vee r) \\[10pt] \frac{\Gamma \Rightarrow \Delta, A \quad B, \Gamma \Rightarrow \Delta}{A \rightarrow B, \Gamma \Rightarrow \Delta} (\rightarrow l) \quad \frac{A, \Gamma \Rightarrow \Delta, B}{\Gamma \Rightarrow \Delta, A \rightarrow B} (\rightarrow r) \\[10pt] \frac{A[t/x], \Gamma \Rightarrow \Delta}{\forall x A, \Gamma \Rightarrow \Delta} (\forall l) \quad \frac{\Gamma \Rightarrow \Delta, A}{\Gamma \Rightarrow \Delta, \forall x A} (\forall r)^* \\[10pt] \frac{A, \Gamma \Rightarrow \Delta}{\exists x A, \Gamma \Rightarrow \Delta} (\exists l)^* \quad \frac{\Gamma \Rightarrow \Delta, A[t/x]}{\Gamma \Rightarrow \Delta, \exists x A} (\exists r) \end{array}$$

* x not free in the conclusion. Weakening: $\frac{\Gamma \Rightarrow \Delta}{A, \Gamma \Rightarrow \Delta}$,

$$\frac{\Gamma \Rightarrow \Delta}{\Gamma \Rightarrow \Delta, A} \quad \text{Cut: } \frac{\Gamma \Rightarrow \Delta, A \quad A, \Gamma \Rightarrow \Delta}{\Gamma \Rightarrow \Delta}.$$

Modal S4 sequent rules ($\Gamma^* = \{\Box B \in \Gamma\}$, $\Delta^* = \{\Diamond B \in \Delta\}$):

$$\begin{array}{c} \frac{A, \Gamma \Rightarrow \Delta}{\Box A, \Gamma \Rightarrow \Delta} (\Box l) \quad \frac{\Gamma^* \Rightarrow \Delta^*, A}{\Gamma \Rightarrow \Delta, \Box A} (\Box r) \\[10pt] \frac{A, \Gamma^* \Rightarrow \Delta^*}{\Diamond A, \Gamma \Rightarrow \Delta} (\Diamond l) \quad \frac{\Gamma \Rightarrow \Delta, A}{\Gamma \Rightarrow \Delta, \Diamond A} (\Diamond r) \end{array}$$

($\Box r, \Diamond l$ discard non- $\Box/\Diamond$ formulas — apply first.)

Modal axioms. K: $\Box(A \rightarrow B) \rightarrow (\Box A \rightarrow \Box B)$ + necessitation $\frac{A}{\Box A}$; T $\Box A \rightarrow A$ (refl); 4 $\Box A \rightarrow \Box \Box A$ (trans); B $A \rightarrow \Box \Diamond A$ (symm). S4=T4, S5=T4B. $\Diamond A \simeq \neg \Box \neg A$.

FOL quantifier equivalences. $\neg \forall x A \simeq \exists x \neg A$; $\neg \exists x A \simeq \forall x \neg A$; pull out if $x \notin B$: $(\forall x A) \wedge B \simeq \forall x (A \wedge B)$, etc.